News Chatbot – Architecture & Code Guide

A real-time news chatbot that answers both current-news questions and general questions. It uses an LLM with tool calling: the model decides when it needs live news, the backend fetches it from GNews, and the model writes a grounded answer with sources.

- **Live frontend:** https://news-chatbot-frontend-nine.vercel.app
- **Live backend:** https://news-chatbot-jbv7.onrender.com
- **Repo:** github.com/jayashreebalinani/News-Chatbot

1. Tech stack

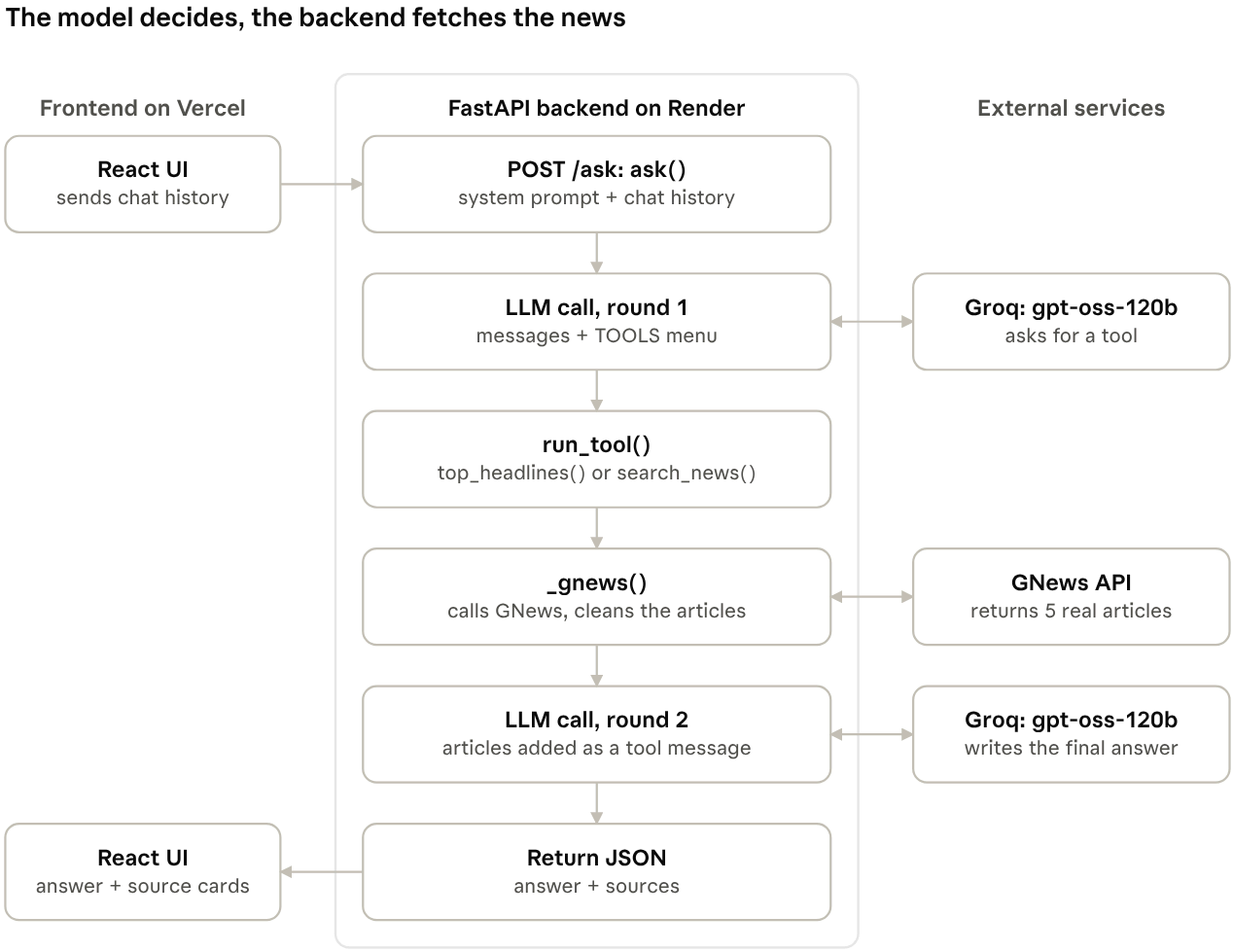
Layer	Technology	Role
Frontend	React + Vite (Tailwind)	Chat UI, keeps conversation history, shows source cards
Backend	Python, FastAPI, Uvicorn	Holds API keys, runs the agent loop, calls tools
LLM host	Groq	Runs the model on fast hardware (OpenAI-compatible API)
LLM	openai/gpt-oss-120b	Decides on tools and writes the final answer
News data	GNews API	Returns real, recent articles
Hosting	Vercel (frontend), Render (backend)	Free-tier deployment

Key idea: Groq is the kitchen, the model is the chef. The model never calls GNews itself. It only asks for a tool by name, and the FastAPI backend runs it.

2. Folder structure

```
News-Chatbot/
├── backend/
|   ├── main.py            # whole FastAPI backend
|   ├── requirements.txt   # packages Render installs
|   └── .env               # keys (never committed)
├── frontend/
|   └── news-chat-app/     # Vite React app (Vercel root directory)
└── .gitignore
```

3. Architecture overview



News chatbot request flow · frontend, backend, external services

Read it top to bottom: the question enters ask(), and every arrow to the right is a call out to Groq or GNews and back.

- **News question:** two LLM calls with one tool call in between.
- **General question:** one LLM call, no tool, answer returned straight away.

4. Request flow example: "Tell me today's latest news"

1. The UI sends the full conversation to `POST /ask`.
2. `ask()` builds the message list: the system prompt first, then the user and assistant history.
3. First LLM call: Groq receives the messages plus the `TOOLS` menu. The model replies with a tool request: `top_headlines(category="general", country="in")`.
4. `ask()` saves that request into the messages, then calls `run_tool()`.
5. `run_tool()` matches the name and calls `top_headlines()`, which calls `_gnews("top-headlines", ...)`.
6. `_gnews()` calls the GNews API, cleans each article down to title, description, source, date and URL, and returns them.
7. The articles are added to the messages with role `tool`, and the sources are collected for the UI.
8. Second LLM call: the model reads the articles and writes a readable answer, citing sources by name.
9. `ask()` returns `{ "answer": ..., "sources": [...] }` and the UI shows the text plus source cards.

5. Backend reference (main.py)

Setup

- `load_dotenv()` reads keys from `.env` locally. On Render, keys come from its Environment Variables.
- `client = Groq(...)` is the connection to Groq, authenticated with `GROQ_API_KEY`.
- `MODEL` is read from the environment, defaulting to `openai/gpt-oss-120b`, so the model can be swapped without code changes.
- `MAX_STEPS = 5` is a safety limit so the agent loop can never run forever.

CORS middleware

`app.add_middleware(CORSMiddleware, allow_origins=[...])` tells the browser which frontend sites may call this backend. It must contain the exact Vercel URL with no trailing slash. Without it, the browser blocks requests and the logs show `OPTIONS /ask 400`.

`_gnews(endpoint, params)`

- **Purpose:** the single place that talks to GNews.
- **Input:** `"search"` or `"top-headlines"`, plus query parameters.
- **Does:** adds language, max 5 results and the API key; retries once on a 429 rate limit; logs status and article count; raises `NewsError` on failure.
- **Output:** a list of clean articles with `title`, `description`, `source`, `publishedAt` and `url`.

`search_news(query)`

- **Purpose:** news about a specific topic, company or person, such as "NVIDIA".
- **Calls:** `_gnews("search", ...)`, sorted by newest.
- **Retry logic:** if nothing is found, it retries with the first two words, then the first word, because GNews requires every word in the query to match.

`top_headlines(category, country)`

- **Purpose:** broad questions like "today's news" or "latest in tech".
- **Calls:** `_gnews("top-headlines", ...)` with a category such as general, business or technology, and a country code, default `in`.

`TOOLS`

A list of descriptions sent to the model, like a menu card: tool name, what it is for, and its input schema. The model reads only this; it never sees the Python code. Good descriptions are what make the model pick the right tool, which is why `search_news` says "1–3 keywords, never a sentence".

`run_tool(name, args)`

A switchboard. The model sends back a tool name as text, and this turns it into a real function call: `top_headlines` or `search_news`. Adding a new tool means adding one line here and one entry in `TOOLS`.

`system_prompt()`

The hidden rulebook, always the first message. It gives today's date, tells the model to use tools for news and answer only from returned articles with source names, and to answer general questions directly.

`Message` and `AskBody` (Pydantic models)

They validate the request body: a list of at least one message, each with role `user` or `assistant` and text content. `model_dump()` converts each validated object back into a plain dictionary for the Groq client.

`error(status, message)`

Returns a clean JSON error instead of crashing, so the UI can show a readable message.

`POST /ask` → `ask(body)`

The main endpoint and the agent loop:

1. Build messages: system prompt, then conversation history.
2. Loop up to `MAX_STEPS`. On the last step, `tool_choice="none"` forces a final answer.
3. If the model returns no tool calls, return the answer and sources.
4. Otherwise, save the model's tool request, run each tool through `run_tool()`, append the results as `tool` messages, collect sources, and loop again.
5. Groq connection, rate-limit and status errors are caught and returned as JSON errors.

`GET /health` → `health()`

Returns `{"status": "ok"}`. Render calls it to check the server is alive.

Message roles

Role	Who	Example
`system`	Developer rules	"Answer only from returned articles"
`user`	Person typing	"Today's news in India"
`assistant`	Model's replies and tool requests	"run top_headlines"
`tool`	Backend returning tool results	The 5 GNews articles

6. Frontend

- Vite React app in `frontend/news-chat-app`.
- Keeps the conversation in state and sends the full history to `/ask` every time, because the model has no memory.
- Reads the backend address from `VITE_API_URL`.
- Shows the answer and source cards with links.
- Never contains any API key.

7. Environment variables

Variable	Where	Purpose
`GROQ_API_KEY`	Backend (.env / Render)	Authenticates with Groq
`NEWS_API_KEY`	Backend (.env / Render)	Authenticates with GNews
`MODEL`	Backend (.env / Render)	Which LLM to use
`VITE_API_URL`	Frontend (Vercel)	Render backend URL, no trailing slash

8. Deployment

Backend on Render

- Web Service from the GitHub repo, branch `main`, free plan.

- Root Directory: `backend`.
- Build: `pip install -r requirements.txt`.
- Start: `uvicorn main:app --host 0.0.0.0 --port $PORT` (Render chooses the port).
- Health check path: `/health`. Auto-deploy on commit is on.
- Free tier sleeps when idle, so the first request after a break is slow.

Frontend on Vercel

- Imported from GitHub, framework preset Vite.
- Root Directory: `frontend/news-chat-app` (where `package.json` lives).
- Install command override: `npm install`.
- Environment variable `VITE_API_URL` set to the Render URL.

9. Problems hit and fixes

Problem	Cause	Fix
`uvicorn` / `python` not found	Packages not installed; macOS uses `python3`	`pip3 install ...`, run `python3 -m uvicorn`
`api_key ... must be set`	`.env` not found or wrong name	`.env` next to `main.py`, exact `GROQ_API_KEY=`
`KeyError: 'topic'`	Wrong request body	Send the keys the endpoint expects
`model_not_found` 404	Groq retired the Llama model	Switch to `openai/gpt-oss-120b`
"Couldn't find news"	Model sent full sentences as GNews query	Keyword-only tool description, `top_headlines` tool, retry with fewer words
`vite: command not found`	Vercel skipped installing packages	Install command override `npm install`
`Could not read package.json`	Wrong Vercel root directory	Root directory `frontend/news-chat-app`

Problem	Cause	Fix
`OPTIONS /ask 400`	CORS origin mismatch or old deploy	Exact Vercel URL in `allow_origins`; wait for Render to show Live

10. Glossary

Each term below has three parts: what it is in plain words, where it shows up in this chatbot, and a one-line way to say it in an interview.

LLM basics

- **LLM (Large Language Model):** a model trained on huge amounts of text that reads text and predicts the next words. It doesn't look things up; it writes from patterns it learned. *In our bot:* gpt-oss-120b reads the conversation and the articles and writes the reply. *Interview line:* "The LLM is the reasoning and writing engine; it doesn't fetch data itself."
- **Model vs. provider:** the model is the brain (gpt-oss-120b, Llama); the provider is the company that runs it on its servers and gives you an API (Groq). *In our bot:* when Llama was retired, we changed only the model name in `.env`; the Groq code stayed the same. *Interview line:* "I kept the model configurable, so switching models was a one-line change."
- **Inference:** running a trained model to get an answer, as opposed to training it. *In our bot:* every `client.chat.completions.create` call is one inference on Groq's hardware.
- **API:** a defined way for one program to ask another for something over HTTP. *In our bot:* the backend uses the Groq API and the GNews API, and the frontend uses our own `/ask` API.
- **API key:** a secret password that proves who is calling an API and who gets billed. *In our bot:* `GROQ_API_KEY` and `NEWS_API_KEY` live only in the backend, never in the browser. *Interview line:* "Keys stay server-side; that's the main reason the frontend never calls Groq directly."
- **Knowledge cutoff:** the date the model's training data stops, so it knows nothing after it. *In our bot:* the model can't know today's news by itself, which is why we fetch it from GNews.

- **Hallucination:** when the model states something false confidently, usually because it's guessing. *In our bot:* without news data, it might invent headlines; the system prompt says answer only from the returned articles.
- **Token:** a small piece of text (roughly ¾ of a word) that models read and bill by. *In our bot:* we trim each GNews article to title, description, source, date and URL, so we send fewer tokens.
- **Stateless:** the model remembers nothing between calls. *In our bot:* the frontend sends the full chat history to `/ask` every time; that's what makes follow-up questions work.

Prompts and messages

- **Prompt:** the text you send the model. *In our bot:* the user's question plus the history and rules.
- **System prompt:** hidden instructions that set the model's behaviour for every reply. *In our bot:* `system_prompt()` gives today's date, says to use tools for news, answer only from articles, and cite sources. *Interview line:* "The system prompt is where I control behaviour, like grounding and citations."
- **Messages and roles:** the conversation is a list of messages, each with a role. `system` = developer rules, `user` = the person, `assistant` = the model's replies and tool requests, `tool` = results the backend sends back. *In our bot:* after GNews returns, `ask()` adds a `tool` message holding the 5 articles.
- **Chat completion:** the API call that takes messages and returns the next reply. *In our bot:* `client.chat.completions.create(model, messages, tools)`, and the text sits in `res.choices[0].message.content`.
- **Grounding:** making the model answer from data you provide, not from memory. *In our bot:* answers about news are grounded in GNews articles, and the sources list proves where each fact came from.

Tool calling and agents

- **Tool calling (function calling):** you give the model a menu of functions; instead of answering, it can reply "please run this function with these inputs". Your code runs it and returns the result. The model never runs code itself. *In our bot:* for "today's news in India", the model asks for `top_headlines` with category `nation` and country `in`. *Interview line:* "The model decides, my code acts."

- **Tool / function schema:** the description of each tool: its name, what it's for, and its inputs as JSON Schema. The model reads only this, so the wording guides its choices. *In our bot:* `TOOLS` says `search_news` takes 1 to 3 keywords, never a sentence; adding that fixed empty search results.
- **`tool_call_id`:** the ID linking a tool request to its result, so the model knows which answer belongs to which request. *In our bot:* every `tool` message carries the ID from the model's request.
- **`tool_choice`:** controls whether the model may call tools. `auto` = it decides, `none` = must answer now. *In our bot:* on the last loop step we set `none` to force a final answer.
- **Agent:** an LLM that can decide on steps and use tools to reach a goal, not just answer once. *In our bot:* the model chooses between answering directly, `top_headlines` or `search_news`.
- **Agent loop:** repeat "call the model, run any tools it asks for, send results back" until it gives a final answer. *In our bot:* the loop in `ask()`, capped at `MAX_STEPS = 5` so it can't run forever.
- **Dispatcher:** code that turns a tool name into a real function call. *In our bot:* `run_tool()` maps the text "top_headlines" to the `top_headlines()` function.
- **RAG (Retrieval-Augmented Generation):** fetch relevant data first, then let the model answer from it. *In our bot:* a simple form of it: fetch news, then answer from it. Our next project does it over PDFs with a vector database.

Backend

- **FastAPI:** a Python framework for building web APIs. *In our bot:* `main.py` defines `/ask` and `/health`.
- **Endpoint:** one URL plus method that does one job. *In our bot:* `POST /ask` answers questions, `GET /health` checks the server is alive.
- **GET vs POST:** GET reads something; POST sends data to be processed. *In our bot:* `/ask` is POST because it receives the chat history in the body.
- **Uvicorn:** the server program that runs a FastAPI app and listens for requests. *In our bot:* `uvicorn main:app --host 0.0.0.0 --port $PORT`.
- **Pydantic and validation:** checking incoming data has the right shape before using it. *In our bot:* `AskBody` rejects a request with no messages or a wrong role; `model_dump()` turns each validated message back into a plain dictionary for Groq.
- **JSON:** the text format APIs use to send data. *In our bot:* `/ask` returns `{"answer": ..., "sources": [...]}`.

- **HTTP status codes:** numbers that say how a request went. 200 = OK, 400 = bad request, 404 = not found, 429 = too many requests, 500 = server crashed. *In our bot:* `404 model_not_found` told us Groq retired Llama; `OPTIONS /ask 400` was a CORS mismatch.
- **Rate limit:** a cap on how many requests you may make in a time window. *In our bot:* `_gnews()` waits one second and retries once on a 429.
- **Middleware:** code that runs on every request before your endpoint. *In our bot:* the CORS middleware checks which site the request came from.
- **CORS:** a browser safety rule: a page on one address can't call a server on another unless that server allows it. The browser first sends an `OPTIONS` "preflight" request to ask. *In our bot:* the backend allows only `https://news-chatbot-frontend-nine.vercel.app`. *Interview line:* "CORS is enforced by the browser; the server just declares which origins it trusts."
- **Environment variables and `.env`:** settings and secrets kept outside the code. `.env` holds them locally and is never committed; hosts store them in their settings. *In our bot:* `GROQ_API_KEY`, `NEWS_API_KEY` and `MODEL` locally in `.env`, and in Render's Environment tab when live.
- **Virtual environment (venv):** a separate set of Python packages per project, so projects don't clash. *In our bot:* `(venv)` in the terminal means its packages are active.
- **`requirements.txt`:** the list of Python packages the app needs, so any machine can install them. *In our bot:* Render runs `pip install -r requirements.txt` before starting.
- **Health check:** a tiny endpoint the host calls to know the app is alive. *In our bot:* `/health` returns `{"status": "ok"}`.
- **Logging:** printing what happened so you can debug. *In our bot:* lines like `GNEWS search ... articles 0` showed the model was sending whole sentences as queries.

Frontend

- **React:** a JavaScript library for building UIs from components and state. *In our bot:* the chat page and its message list.
- **Vite:** a fast build tool that turns React code into plain HTML, CSS and JavaScript. *In our bot:* Vercel runs `vite build` and serves the output.
- **`VITE_` variables:** Vite only exposes environment variables starting with `VITE_` to the browser, so never put secrets in them. *In our bot:* `VITE_API_URL` holds the public Render URL.

Data source

- **GNews API:** a news service that returns recent articles as JSON. *In our bot:* `search` for a topic like NVIDIA, `top-headlines` for broad questions like today's news.
- **Keyword query:** a short search like "NVIDIA", not a sentence. GNews needs every word to match, so sentences return nothing. *In our bot:* `search_news` retries with fewer words when nothing comes back.

Deployment

- **Deployment:** putting the app on a server so anyone can use it through a public URL.
- **Render:** a hosting platform that runs backend services from your GitHub repo. *In our bot:* runs FastAPI on the free plan.
- **Vercel:** a hosting platform built for frontends. *In our bot:* builds and serves the Vite app.
- **Build vs start command:** build prepares the app once (install packages); start runs it. *In our bot:* build `pip install -r requirements.txt`, start `uvicorn ...`.
- **`0.0.0.0` and `$PORT`:** `0.0.0.0` lets the server accept outside connections; `$PORT` is the port the host assigns. *In our bot:* without them, Render can't reach the app.
- **Root directory:** the folder inside the repo the host should treat as the app. *In our bot:* `backend` on Render, `frontend/news-chat-app` on Vercel, the fix for `Could not read package.json`.
- **Auto-deploy (CI/CD):** each push to GitHub triggers a new build and release automatically. *In our bot:* pushing the CORS fix redeployed Render by itself.
- **Cold start:** free servers sleep when idle, so the first request is slow while it wakes up. *In our bot:* the first question after a break can take up to a minute.
- **`.gitignore`:** a list of files Git must never upload. *In our bot:* keeps `.env`, `venv/`, `node_modules/` out of GitHub.

Docker (next step)

- **Docker:** a tool that packages an app with everything it needs, so it runs the same on any machine.
- **Dockerfile:** the recipe: base Python version, install packages, copy code, start command. *For our bot:* `FROM python`, `pip install -r requirements.txt`, then `uvicorn main:app`.

- **Image vs container:** the image is the built, frozen box; a container is that image running. *Interview line:* "An image is like an installer, a container is the app running from it."

11. Next steps

- Streaming responses so text appears word by word.
- Structured JSON output for each story.
- Dockerfile for the backend, and running Render from Docker.